

plan

Flatcar VEX Automation — Design Plan

Companion to [current-state.md](#) (the as-is system). This is a **phased roadmap** — not a single fixed architecture. **Phase 1 is a proof of concept**: deliberately narrow, meant to prove the core mechanic (release → VEX) works before anything more ambitious is committed to. **Everything past Phase 1** is grouped under “Future Phases” for now — that bucket may end up being one phase or several (Phase 2, Phase 3, ...); it isn’t scoped or sequenced yet, just a holding area for candidates and open tensions so they aren’t lost.

Status: Phase 1 (PoC) design agreed, implementation not yet started. Future Phases are discussion-only — not committed, not sequenced, not necessarily “Phase 2” as a single next step.

Contents:

Phase 1 — PoC (committed scope): [1. Goals & Non-Goals](#) · [2. Decision Log](#) · [3. PoC Scope](#) · [4. Architecture](#) · [5. Rolling-Forward Mechanic](#) · [6. Tech Stack & Repo Layout](#)

Future Phases (not committed, not sequenced): [7. Future Phase Candidates](#) · [8. Human-Override Loop](#) · [9. Open Questions](#) · [10. Prior Art / Feature-Creep Check](#)

PHASE 1 — Proof of Concept (Committed Scope)

Everything in sections 1–6 below is the agreed, in-progress **PoC** design — deliberately the smallest slice that proves the core release → VEX mechanic end-to-end. It is not meant to be feature-complete or production-final; it’s meant to be small enough to ship and learn from before deciding what comes next. Nothing here is speculative.

1. Goals & Non-Goals

Goal (PoC): Produce a machine-readable, spec-compliant **OpenVEX** document for every released Flatcar version (all channels, including patch releases), derived only from data Flatcar has already published — no speculative “affected” claims yet. The point of this phase is to prove the release → VEX mechanic works end-to-end on real data, not to cover every case.

Explicit non-goals for the PoC (deferred, not rejected — see the Future Phases §7 for these as candidates):

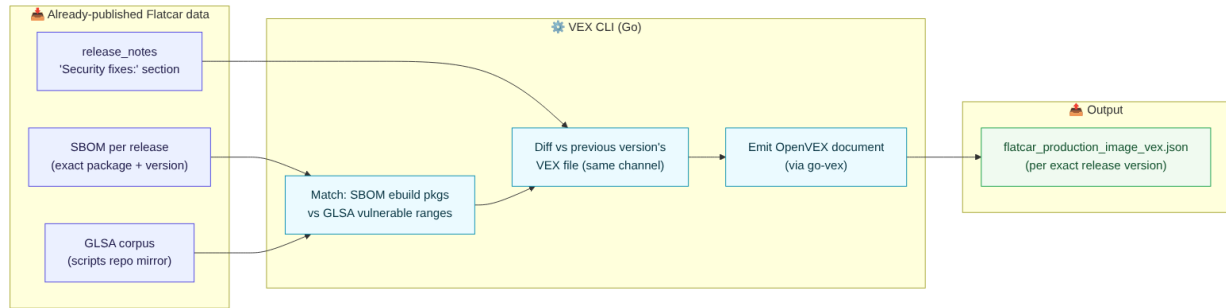
- No backward historical backfill / no attempt to determine exactly when a CVE started affecting older releases.
- No AI-assisted triage of new GLSAs into GitHub issues (the “Slow Path” from the original sketch) — that’s a separate, later phase.
- No coverage of non-ebuild packages (golang/rust modules in the SBOM) yet — GLSA-based matching only for the PoC (see Future Phases candidate #1, OSV.dev).
- No decision yet on public hosting/publishing location for the resulting VEX files (see Open Questions #2).
- No general-purpose binary/filesystem scanning or component-discovery of our own — this project consumes Flatcar’s already-published SBOM as its ground truth (see §10 for how this differs from a scanner like `cve-bin-tool`).

2. Decision Log

#	Decision	Rationale
1	PoC only covers current + new releases , no historical backfill	Avoids the unresolved question of “how far back was a CVE actually exploitable” — sidestepped entirely by never looking backward.
2	Ground truth = already-released data only : SBOM (exact package+version) + GLSA (vulnerability ranges) + published security changelog text	Nothing here is inferred/guessed; every VEX statement traces to a fact Flatcar already published.
3	One VEX file per exact release version (not one monolithic file, not one per minor line)	Mirrors the existing per-version SBOM publishing pattern; matches standard supply-chain practice of colocating VEX with its artifact; keeps each file self-contained and independently fetchable.
4	Every release matters, including patch releases	Patch releases are the primary CVE-fix vehicle (per Dongsu, most fixes land via routine bumps) — coarser granularity would miss the exact events this system exists to track.

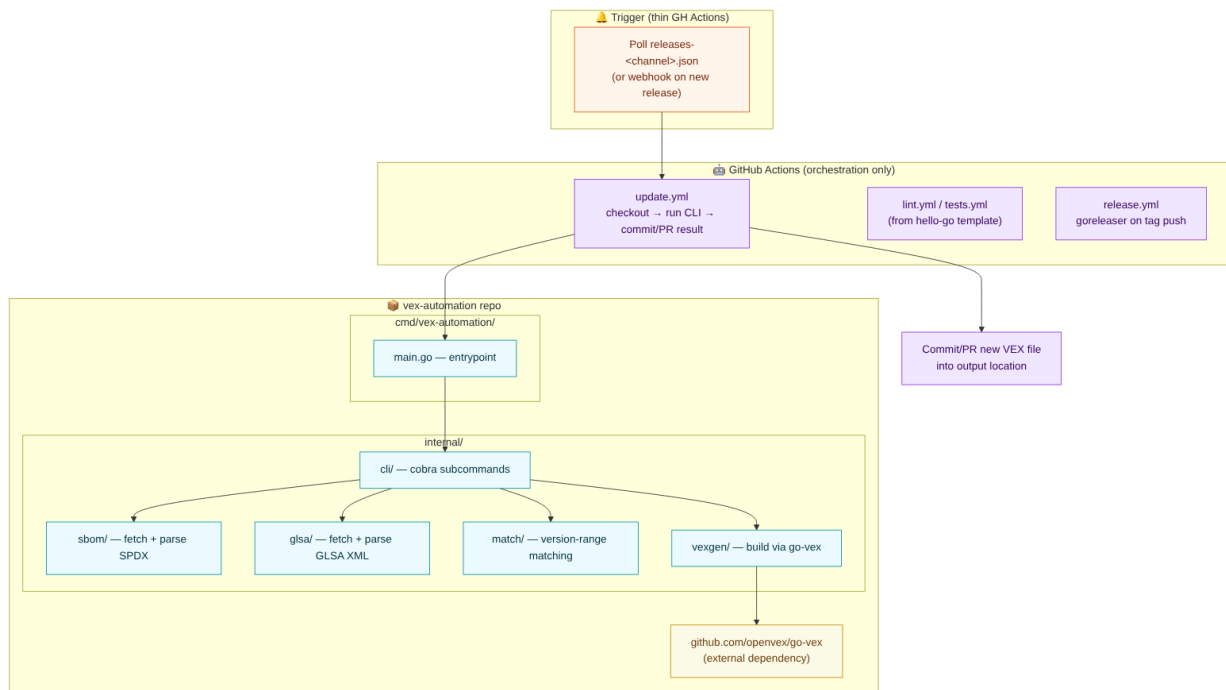
#	Decision	Rationale
5	Rolling-forward model: bootstrap once from each channel's current head, then carry state forward release-to-release (diff against the immediately preceding version)	Avoids needing full history; naturally self-correcting as new data arrives.
6	Language: Go	OpenVEX's reference implementation (<code>go-vex</code> , <code>vexctl</code>) is Go, maintained by the spec authors under the official <code>openvex</code> GitHub org. Python's only option (<code>vexipy</code>) is an unofficial, single-maintainer reimplementation outside that org. Using <code>go-vex</code> gives spec-correctness, validation, and merge support for free.
7	Architecture: CLI tool + thin GitHub Actions orchestration , not logic embedded in workflow YAML	Matches existing Flatcar convention (<code>show-fixed-kernel-cves.py</code> , <code>sync_with_gentoo.sh</code> are real scripts called from workflows). Keeps matching/diffing/VEX-generation logic unit-testable, locally runnable, and versionable independent of any CI trigger.
8	Project scaffold: adapt the hello-go template	Modern Go 1.24+ tool-dependency pattern, cobra CLI, clean <code>cmd/+internal/</code> layout, working lint/test/release CI already wired up — solid base to extend rather than build from scratch.
9	License: Apache-2.0 (already resolved, not open)	The <code>vex-automation</code> repo scaffold already ships an Apache-2.0 LICENSE file. hello-go 's own MIT license needs to be dropped when adapting the template — the repo's existing Apache-2.0 wins, and it also matches <code>go-vex</code> 's license.

3. PoC Scope



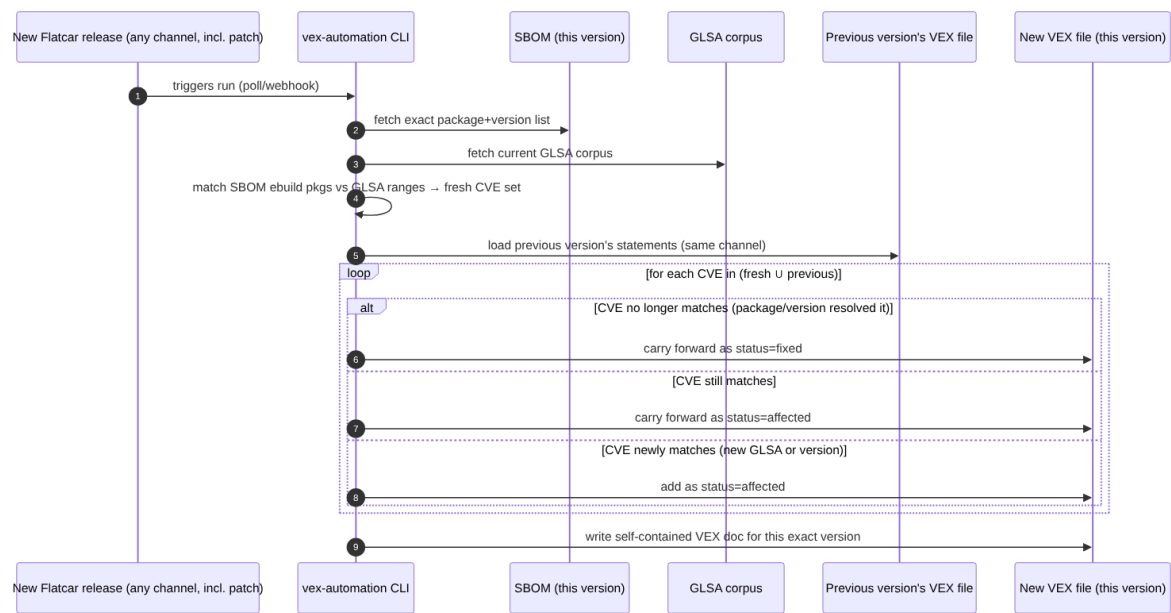
diagram

4. Architecture



diagram

5. Rolling-Forward Mechanic



diagram

Bootstrap case (first run per channel): there is no “previous version” — the CLI runs once against each channel’s *current* head only, with an empty previous state, producing the initial affected set to roll forward from. No history before that point is touched.

6. Tech Stack & Repo Layout

Scaffold reference: [John15321/hello-go](https://github.com/John15321/hello-go)

Concern	Choice	Source
Language	Go	Decision Log #6
CLI framework	spf13/cobra	via hello-go template
VEX generation	github.com/openvex/go-vex	official OpenVEX reference implementation
Lint	golangci-lint (errcheck, govet, staticcheck, unused, misspell, revive, gocritic)	hello-go .golangci.yml
Test runner	gotestsum (race + coverage)	hello-go Makefile
Release	goreleaser (cross-compiled linux/darwin/windows x	hello-go .goreleaser.yaml

Concern	Choice	Source
	amd64/arm64, triggered on tag push)	
Dev tooling	Go 1.24+ <code>tool</code> directives in <code>go.mod</code> — no global installs	hello-go template
CI orchestration	Thin GitHub Actions (<code>lint.yml</code> , <code>tests.yml</code> , <code>release.yml</code> reused as-is; new <code>update.yml</code> for the actual VEX pipeline)	hello-go + this plan

Planned repo layout (adapted from [hello-go](#)):

```
vex-automation/
├─ cmd/vex-automation/main.go      entrypoint
├─ internal/
│  └─ cli/                          cobra subcommands (bootstrap, update, match,
... )
│  └─ sbom/                         fetch + parse SPDX SBOM
│  └─ glsa/                         fetch + parse GLSA XML corpus
│  └─ match/                        package/version-range matching logic
│  └─ vexgen/                       build OpenVEX docs via go-vex
├─ docs/
│  └─ current-state.md              (existing)
│  └─ plan.md                       (this file)
└─ .github/workflows/
   └─ lint.yml / tests.yml          (from hello-go, reused)
   └─ release.yml                  (from hello-go, reused)
   └─ update.yml                   (new – the actual VEX-generation trigger)
```

FUTURE PHASES — Candidates (Not Committed, Not Sequenced)

Everything in sections 7–9 below is discussion/brainstorming only — **nothing here is agreed, scheduled, or numbered as a specific next phase**. This may become one phase or several; it's a holding area for candidates and open tensions surfaced while designing the

7. Future Phase Candidates (not committed, not sequenced)

#	Candidate	Why
1	OSV.dev integration for non-ebuild packages	SBOM's golang purls (1070 of ~1400 packages) have zero GLSA coverage today. OSV.dev's API is purl-native (<code>{"package": {"purl": "pkg:golang/...@v1.2.3"}},</code> batchable) and already aggregates Go vulndb, GHSA, RustSec, PyPI Advisory DB — one integration instead of many.
2	Kernel CVE feed (<code>lore.kernel.org/linux-cve-announce</code>) as its own source	Kernel CVEs largely have no formal GLSA; this feed already exists and is proven (used by <code>show-fixed-kernel-cves.py</code>) but currently only feeds changelog text, not VEX.
3	Human-override loop — connect the existing manual advisory issues (Path C) to VEX	See §8 below — the key mechanism for correcting “assumed affected” GLSA/OSV matches once a maintainer determines Flatcar isn't actually exploitable.
4	Real-time trigger via GLSA RSS feed (<code>security.gentoo.org/glsa/feed.rss</code> , verified live)	Decouples “affected” detection from Flatcar's own monthly rsync cadence — closer to the original plan's “Fast Path,” updates between releases rather than only at release time.
5	Combined rollup VEX file	Aggregated “all channels/versions” feed for consumers (e.g. NVD) who want one fetch — generated <i>from</i> the per-release files, not a new source of truth.

8. Human-Override Loop (detail)

Problem: the PoC's GLSA/OSV matching produces `affected` statements purely from version-range matching. Per Dongsu, a considerable number of these don't actually affect Flatcar in practice (wrong build config, code path never executed, etc.) — today a human resolves this by hand in the existing manual advisory issues (Path C), but that resolution never reaches the VEX file.

Proposed mechanism: - A small labeling convention on top of the *existing* advisory issue process (e.g. `vex/not-affected`, `vex/affected`, `vex/fixed`), applied **per CVE**, since one issue can bundle multiple CVEs with different resolutions (confirmed via issue #2188). - **Dual trigger**, mirroring the existing GLSA sync + build-gate redundancy pattern already used elsewhere in this project: - Event-driven (`on: issues: types: [labeled, closed]`) for near-real-time updates. - Scheduled reconciliation cron as a safety net for anything the webhook missed. - Maps to OpenVEX's `not_affected` status with an explicit `justification` (one of the 5 spec-defined values, e.g. `vulnerable_code_not_in_execute_path` from the original sketch). - **Integrates into the rolling-forward mechanic (§5)** as a new authoritative input: when computing a CVE's status for a release, check for a human override *first*, falling back to raw SBOM/GLSA/OSV matching only if none exists.

⚠ **Open tension — VEX document immutability is unresolved, not decided:** A human override may need to correct a CVE's status across **every already-published per-release VEX file** where it appears, not just new ones going forward. We discussed this and leaned toward “patch retroactively” for accuracy, but that conflicts with the general expectation that published VEX documents (like SBOMs) are immutable release artifacts — a consumer who cached an older file would be silently out of date with no signal, unless every retroactive patch also bumps that document's `version`/timestamp fields so re-fetching is detectable. **This tradeoff (accuracy vs. immutability) is explicitly still up for debate** and needs a real decision — including whether “retroactive patch” is even acceptable to downstream consumers — before any future-phase implementation, not just a default assumption.

9. Open Questions

These are deliberately unresolved — flagged for a future decision, not blocking PoC design:

1. **Product identifier scheme** for OpenVEX `product` field (purl vs CPE, and exact format for “this exact Flatcar release on this channel/arch”).
2. **Publishing location** for generated VEX files — colocated with release artifacts (needs release-pipeline integration) vs. hosted separately (e.g. GitHub Pages / repo artifact keyed by

version) for this first iteration.

3. **Combined rollup file** — whether/when to generate an aggregated “all channels/versions” VEX document from the per-release files, for consumers like NVD that may want one feed.
 4. **Non-ebuild package coverage** — see Future Phases candidate #1 (OSV.dev).
 5. **Config loading** — flags-only for the PoC, or introduce a config file once inputs (GLSA path, SBOM URL template, output path) multiply.
 6. **AI-assisted triage / GitHub issue integration** (“Slow Path” from the original sketch) — explicitly deferred; this plan covers only the release → VEX sync.
 7. **VEX document immutability vs. retroactive correction** — see §8. Unresolved: whether published per-release VEX files should ever be rewritten after the fact, and if so, how consumers are meant to detect the change.
 8. **Multi-source CVE deduplication/precedence** — see §10’s coverage diagram. When the same CVE for the same package is reported by more than one source (e.g. Gentoo happens to GLSA a Rust dependency that also has a GitLab Advisory Database entry, or a kernel CVE appears both via the official feed and gets independently discussed on `oss-security`), the tool must emit **one** canonical VEX statement per CVE+product, not one per source. Unresolved: which source wins if their `status/justification` ever disagree (e.g. GLSA implies `affected` but a human override from Path C says `not_affected`) — a source precedence ordering needs to be defined, not left as “last write wins” by accident of processing order.
-

10. Prior Art / Feature-Creep Check

Two existing open source tools sit near this problem space. Both were checked directly against their real docs/READMEs/repos (not assumed from memory) before writing this section, specifically to avoid re-describing a solved problem.

OpenCVE (`opencve/opencve`)

What it actually is: a CVE **monitoring and notification platform**. You subscribe to vendors/products, it cross-references MITRE/Vulnrichment/NVD/RedHat, and it notifies you (email/webhook/Slack) when a subscribed CVE appears or changes. It also gives each subscribed CVE a human-assignable status (`under analysis`, `risk accepted`, `to be verified`, ...) and lets you tag/organize/report on them.

Confirmed absence of overlap: OpenCVE has **no SBOM ingestion and no VEX production/consumption of any kind** — this was verified directly against its own docs (`docs.opencve.io`) feature list, which covers `explore/subscribe/organize/notify/tag/report` only. (An earlier AI web-search summary in this project’s research had speculated OpenCVE “generally” supports VEX — that claim is **not supported by OpenCVE’s actual documentation** and should

be disregarded.) OpenCVE answers “which CVEs affect the vendors/products I follow, and what’s my team’s triage status on them” — a fundamentally different question from “what is the machine-readable, per-release, exploitability statement for this exact Flatcar image.” **No feature-creep risk here** — different problem, no functional overlap.

CVE Binary Tool (ossf/cve-bin-tool)

What it actually is: a scanner that finds known-vulnerable components either by binary-scanning a filesystem (448 built-in checkers) or by **consuming an existing SBOM** (SPDX, CycloneDX, or SWID). It matches found components against multiple vulnerability databases — **NVD, RedHat, OSV.dev, GitLab Advisory Database, and the curl project’s own vuln feed** — and can then:

- **Generate a VEX** from the scan, in **CSAF, CycloneDX, or OpenVEX** format (`--vex-type {csaf,cyclonedx,openvex} --vex-output <file>`)
- **Consume a VEX file as a triage database** (`--vex-file <file>`) so a human’s prior not_affected/affected/accepted-risk decisions persist across future scans instead of re-triaging the same CVE every run — explicitly designed to be shared “across different runs of cve-bin-tool or with other tools that support CycloneDX VEX, OpenVEX and CSAF format.”

This is the real overlap to be honest about. The generic shape — *SBOM* → *match against vuln sources* → *emit VEX* → *let a human edit/annotate it* → *feed that back in so future runs respect the human’s call* — is structurally the same loop as this project’s PoC (§3–§5) and the Future Phases Human-Override Loop (§8). cve-bin-tool already does this, is OSS, Python/GPL-3.0, has an official GitHub Action, and its OSV.dev data source already covers the exact non-ebuild/Go-ecosystem gap flagged as Future Phase candidate #1.

Why we are not just adopting it as-is, and where the real gap remains:

Gap vs. cve-bin-tool	Why it matters for Flatcar specifically
No Gentoo GLSA awareness ¹	cve-bin-tool’s sources are NVD/RedHat/OSV/GitLab/curl — none of these is GLSA. Flatcar’s build-gate (glsa-check) and its ~3.8k-file GLSA mirror are the one detection mechanism Flatcar already trusts and blocks builds on (current-state.md §3); a generic tool with no GLSA input would silently miss that entire axis.
No concept of a Flatcar release/channel identity	cve-bin-tool’s VEX output describes whatever it scanned in that run; it has no built-in notion of “Alpha/Beta/Stable/LTS channel, this exact version, roll forward from the previous release’s VEX” (§5, Rolling-Forward Mechanic) — that continuity logic

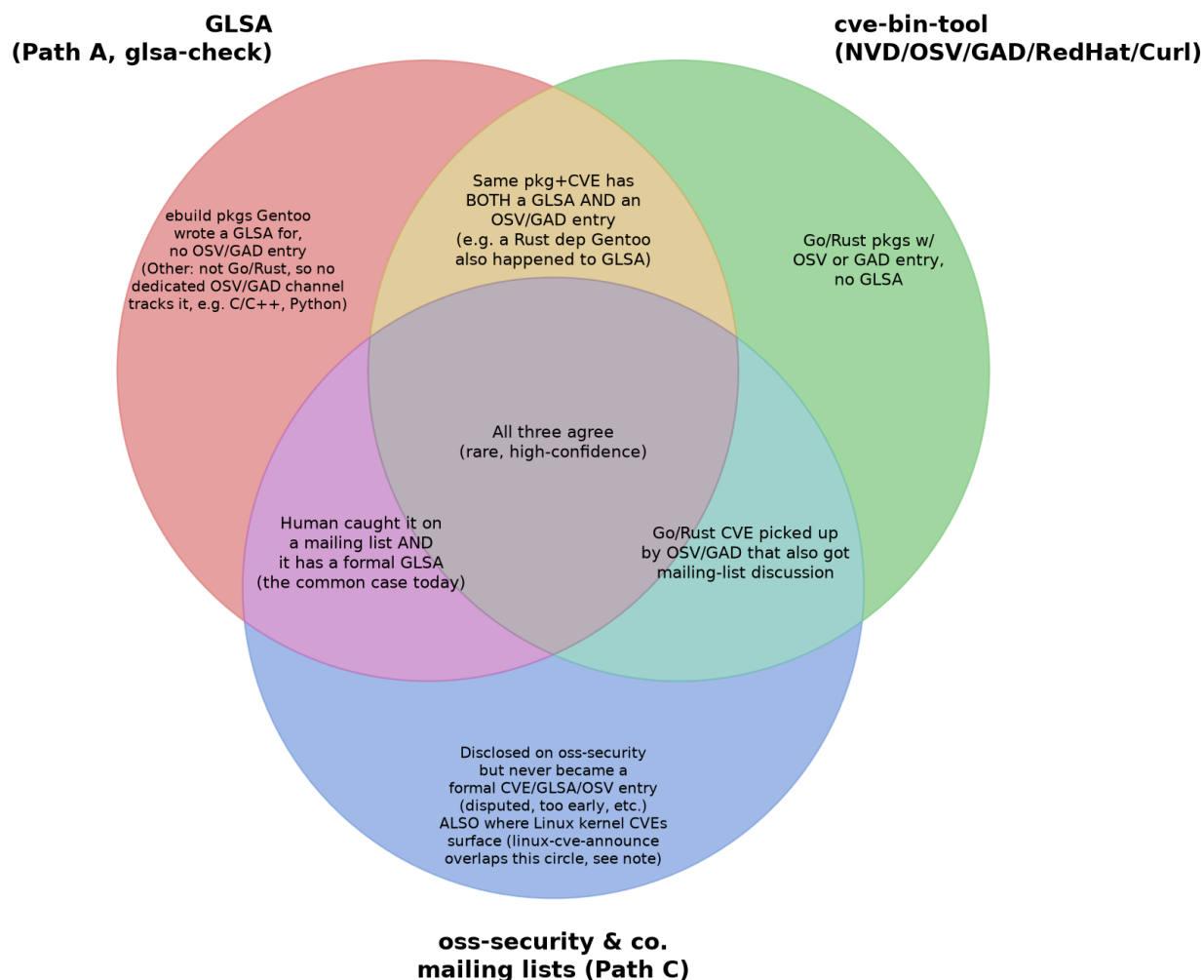
Gap vs. cve-bin-tool	Why it matters for Flatcar specifically
	is Flatcar-specific and would need to be built around it either way.
No native tie-in to Flatcar’s existing manual advisory process	The Human-Override Loop (§8) is designed to consume Flatcar’s <i>existing</i> advisory-labeled GitHub issues (Path C) as the human-decision source of truth — cve-bin-tool’s triage loop expects a human to hand-edit its own VEX/JSON file directly, not read from an external issue tracker.
Distribution-image scope, not source/binary scan scope	cve-bin-tool’s core detection is “scan this filesystem/binary/SBOM for known-vulnerable components.” Flatcar’s ground truth is narrower and stronger: the SBOM’s <code>ebuild</code> purls already give an exact package+version tied to a signed, released artifact (current-state.md §6) — matching still needs to happen, but the input is authoritative already, not discovered by a generic scanner.

Verdict: cve-bin-tool is the closest prior art found, and is worth revisiting concretely in a future phase — either as a **library/data-source dependency** (e.g. calling into its OSV.dev-backed matching instead of hand-rolling that integration, per Future Phase candidate #1) or as a **reference implementation** for the human-triage-file pattern in §8. It is not, however, a drop-in replacement for the PoC: it has no GLSA input, no Flatcar release/channel model, and no tie-in to Flatcar’s existing advisory-issue workflow — all three are Flatcar-specific integration work this project exists to do. Building a thin, Flatcar-aware layer is judged **not** feature creep; re-implementing a general-purpose binary/SBOM vulnerability scanner from scratch (checkers, multi-source dedup, etc.) *would* be — and is explicitly out of scope (see §1 Non-Goals).

¹ Double-checked directly, not assumed: cve-bin-tool’s `doc/MANUAL.md` data-source list is exactly `{NVD, OSV, GAD, REDHAT, CURL}` (its `--disable-data-source` flag enumerates the same 5). Its OSV.dev dependency doesn’t cover the gap either — OSV.dev’s own live ecosystem list (`storage.googleapis.com/osv-vulnerabilities/ecosystems.txt`) has no “Gentoo” entry, and Gentoo/GLSA integration into OSV.dev is an open, unresolved community request as of this writing, not an existing feature.

Where the sources actually overlap

ILLUSTRATIVE ONLY — not proportional to real CVE counts.
Real overlap sizes require running actual matching against Flatcar's SBOM.



Note: Linux kernel CVEs (lore.kernel.org/linux-cve-announce, our source D4) are not one of these three circles, but they frequently surface on oss-security too before/alongside that feed, so kernel issues live inside the oss-security-only wedge above, not in a fourth circle.

Illustrative Venn diagram of GLSA, cve-bin-tool, and oss-security mailing-list CVE coverage overlap

⚠ Illustrative only — not proportional to real CVE counts. The three-circle shape and equal region sizes are for communicating the *categories* of overlap, not measured data. Getting real proportions would require actually running GLSA-matching and OSV/GAD-matching against Flatcar's SBOM and counting real intersections — a natural validation step once the PoC's matcher exists, not something we can produce from docs alone today.

The concrete case that motivates this diagram: **a single Rust dependency with a known CVE can legitimately be reported by more than one source at once** — e.g. Gentoo happens to write a GLSA for it *and* it already has a GitLab Advisory Database (`cargo`) entry that cve-bin-tool would also surface. Without deduplication, a naive “run every source and union the results” approach

would emit **two separate VEX statements for the same CVE+product**, which is invalid/confusing for a consumer. This is now tracked explicitly as Open Question #8 (§9): sources must be deduplicated by canonical CVE ID per package, and a precedence order defined for when sources disagree on status.

A fourth wrinkle: Linux kernel CVEs aren't a clean fourth circle, they live inside oss-security's wedge. Our D4 source (lore.kernel.org/linux-cve-announce, the kernel's own CNA announcement feed since 2024) is *not* independent of the oss-security mailing list: kernel security issues are frequently disclosed and discussed on oss-security first or in parallel, before/alongside the dedicated kernel feed. So rather than drawing a fourth overlapping circle (which `matplotlib-venn` can't render cleanly for 4+ sets anyway), the diagram calls this out as a note: kernel CVEs sit inside the oss-security-only wedge, and D4 should be treated as a narrower, more-structured subset of what oss-security already covers for the kernel, not a wholly separate population. This reinforces Open Question #8: the dedup/precedence logic needs to account for D3 (oss-security) and D4 (kernel feed) frequently reporting the *same* kernel CVE.